

Adaptive Document Preparation System

A quiz engine that reads a document, tests you, and gets smarter from every mistake you make.

10 sections · 133 sub-sections · 195 chunks · 69 topics · 4-layer validation

1 Pick topics — you choose which sections to study

2 Find material — search the knowledge base + recall your weak spots

3 Write questions — AI drafts MCQs (Groq → Gemini)

4 Fact-check — reject anything not in the document

5 Score & learn — grade you, remember what you missed ↴

Python

LangGraph

PostgreSQL

ChromaDB

FastAPI

Docker

swipe →

Two storage engines — and why

Each store answers a different question. Mixing them into one would do both jobs badly.

PostgreSQL

relational source of truth — 11 tables, 4 layers

STRUCTURE

sections · chunks

TOPIC GRAPH

topics · section_topics ·
chunk_topics · question_topics

SESSIONS

sessions · questions · answers

MASTERY = MEMORY

topic_mastery ·
section_topic_mastery

ChromaDB

vector index — semantic ranking only

195 chunk embeddings, **384-dim** (MiniLM)

HNSW index · **cosine** similarity

metadata for filtering:

section_id · chunk_kind · pages

No business logic — it only knows what text is *relevant*.



sub_section_id — same key in BOTH stores

FK policy: CASCADE almost everywhere, but **RESTRICT** on questions.section_id — you can't delete a section while history still points to it.

Dual retrieval: relevant × personal

Two lanes run in parallel, then fuse into one prompt.

CONTENT

“what is there to ask about?” · reads Chroma
→ Postgres

- ▶ embed query → **ChromaDB HNSW** search
- ▶ filter by section, drop glossary noise
- ▶ top-k ids → fetch rows from **Postgres**, **keep Chroma's ranking**
- ▶ resolve §X.Y cross-references (depth 1)
- ▶ token-budget filter

HISTORY

“what is this user weak at?” · reads Postgres mastery

- ▶ read **mastery tables** + topic depth
- ▶ **Allocator**: $\text{weight} = \text{global} \times \text{section} \times \text{depth}$
- ▶ → per-topic seeds (weakest get more)
- ▶ build **WEAK / MASTERED** summary
- ▶ token-budgeted (3-level compression)

→ **Prompt assembly (LangChain)**: relevant chunks + weak-spot seeds + WEAK/MASTERED summary + JSON schema

Why two? A vector DB can't tell you which topic a user keeps failing; relational tables can't do fuzzy semantic search. One finds *relevant* material, the other makes it *personal*.

Generate, then guard the output

An LLM call is cheap; trusting it blindly is expensive. Every question runs a gauntlet.

LLM gateway: Groq (primary) → Gemini (fallback) via `.with_fallbacks()`. One API failure never breaks a session. TPM pacing between calls.

LAYER 1

Schema

Pydantic + JSON parser — strips fences, validates shape

LAYER 2

Unique choices

4 distinct options, no repeats

LAYER 3

Hallucination

the cited `source_quote` must fuzzy-match the retrieved text — else reject

LAYER 4

Near-duplicate

SequenceMatcher ≥ 0.65 vs recent questions

Self-healing: if validation drops questions below target, the LangGraph loop runs `validate` → `generate` again (bounded).

Closing the adaptive loop

This is what makes it “adaptive” and not just a quiz generator.



Every answer updates a mastery weight — wrong raises it, a winning streak decays it (mastered)



Regression guard — a mastered topic answered wrong is un-mastered & bumped back up



Next session reads the new weights — so retrieval & questions shift toward your weak spots

🔄 **mistake → weight → smarter next question. The memory is persisted in PostgreSQL, not held in RAM.**

PrepState (TypedDict)

in-run working memory — one session

PostgreSQL

long-term memory — across all sessions

The hard parts were the point

Building something that *works once* is an afternoon. Building something that *adapts* is a different discipline.

Hard: dual-store consistency via a shared key

Hard: rank-preserving re-order across stores

Hard: cross-reference graph walk

Hard: proportional allocation with per-topic guarantees

Hard: regression detection on mastered topics

Hard: token-budgeted history with graceful compression

Hard: grounding every question in the source text

Python

LangChain

LangGraph

PostgreSQL

ChromaDB

FastAPI

Docker

Groq

Gemini

Full code, schema & a runnable demo are on GitHub.

 github.com/istiak133/Adaptive_RAG